

# TREE 2

## PROBLEM 6

### QUESTION:

| DS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Session | Tree 2 | Question Information | Level 1 | Challenge 76 |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------|----------------------|---------|--------------|
| <b>Question description</b> <p>Your task is to maintain an array of <math>n</math> values and efficiently process the following types of queries:</p> <ol style="list-style-type: none"><li>1. Increase the first value in range <math>[a,b]</math> by 1, the second value by 2, the third value by 3, and so on.</li><li>2. Calculate the sum of values in range <math>[a,b]</math>.</li></ol> <b>Input</b> <p>The first input line has two integers <math>n</math> and <math>q</math>: the size of the array and the number of queries.</p> <p>The next line has <math>n</math> values <math>t_1, t_2, \dots, t_n</math>: the initial contents of the array.</p> <p>Finally, there are <math>q</math> lines describing the queries. The format of each line is either "<math>1 \ a \ b</math>" or "<math>2 \ a \ b</math>".</p> <b>Output</b> <p>Print the answer to each sum query.</p> <b>Constraints</b> <ul style="list-style-type: none"><li><math>1 \leq n, q \leq 2 \cdot 10^5</math></li><li><math>1 \leq t_i \leq 10^6</math></li><li><math>1 \leq a \leq b \leq n</math></li></ul> |         |        |                      |         |              |

#### Logical Test Cases

| Test Case 1                                 |
|---------------------------------------------|
| INPUT (STDIN)                               |
| 5 3<br>3 7 2 2 5<br>3 2 3<br>1 3 6<br>1 2 3 |
| EXPECTED OUTPUT                             |
| 9                                           |

| Test Case 2                                 |
|---------------------------------------------|
| INPUT (STDIN)                               |
| 5 3<br>4 2 3 1 7<br>2 1 5<br>1 1 5<br>2 1 5 |
| EXPECTED OUTPUT                             |
| 17<br>32                                    |

#### Mandatory Test Cases

| Test Case 1                      |
|----------------------------------|
| KEYWORD                          |
| void build(int k,int<br>l,int r) |

#### Complexity Test Cases

| Test Case 1           |
|-----------------------|
| CYCLOMATIC COMPLEXITY |
| 2                     |

| Test Case 2 |
|-------------|
| TOKEN COUNT |
| 800         |

| Test Case 3 |
|-------------|
| NLOC        |
| 71          |

## SOURCE CODE:

```
#include <stdio.h>

#define MAX_CAP 200005

long long tree_sum[4 * MAX_CAP];
long long pending_a[4 * MAX_CAP];
long long pending_d[4 * MAX_CAP];
long long base_arr[MAX_CAP];

void construct_seg(int idx, int low, int high) {
    pending_a[idx] = 0;
    pending_d[idx] = 0;
    if (low == high) {
        tree_sum[idx] = base_arr[low];
        return;
    }
    int mid = (low + high) / 2;
    construct_seg(2 * idx, low, mid);
    construct_seg(2 * idx + 1, mid + 1, high);
    tree_sum[idx] = tree_sum[2 * idx] + tree_sum[2 * idx + 1];
}

void apply_progression(int idx, int low, int high, long long a, long long d) {
    long long span = high - low + 1;
    tree_sum[idx] += a * span + d * span * (span - 1) / 2;
    pending_a[idx] += a;
    pending_d[idx] += d;
}

void propagate_down(int idx, int low, int high) {
    if (pending_a[idx] == 0 && pending_d[idx] == 0) return;
    int mid = (low + high) / 2;
    apply_progression(2 * idx, low, mid, pending_a[idx], pending_d[idx]);
    long long r_start = pending_a[idx] + pending_d[idx] * (mid - low + 1);
    apply_progression(2 * idx + 1, mid + 1, high, r_start, pending_d[idx]);
    pending_a[idx] = 0;
    pending_d[idx] = 0;
}

void range_add_ap(int idx, int low, int high, int ql, int qr) {
    if (ql <= low && high <= qr) {
        long long a = low - ql + 1;
        apply_progression(idx, low, high, a, 1);
        return;
    }
    propagate_down(idx, low, high);
    int mid = (low + high) / 2;
    if (ql <= mid) range_add_ap(2 * idx, low, mid, ql, qr);
    if (qr > mid) range_add_ap(2 * idx + 1, mid + 1, high, ql, qr);
    tree_sum[idx] = tree_sum[2 * idx] + tree_sum[2 * idx + 1];
}

long long range_calc_sum(int idx, int low, int high, int ql, int qr) {
    if (ql <= low && high <= qr) return tree_sum[idx];
    propagate_down(idx, low, high);
    int mid = (low + high) / 2;
    long long total = 0;
    if (ql <= mid) total += range_calc_sum(2 * idx, low, mid, ql, qr);
    if (qr > mid) total += range_calc_sum(2 * idx + 1, mid + 1, high, ql, qr);
    return total;
}

int main() {
    int length, op_count;
    if (scanf("%d %d", &length, &op_count) != 2) return 0;

    for (int i = 1; i <= length; i++) scanf("%lld", &base_arr[i]);
    construct_seg(1, 1, length);

    while (op_count--) {
        int type, l, r;
        scanf("%d %d %d", &type, &l, &r);
        if (type == 1) {
            range_add_ap(1, 1, length, l, r);
        } else {
            printf("%lld\n", range_calc_sum(1, 1, length, l, r));
        }
    }
    return 0;
}
```

## OUTPUT:

```
5 3
4 2 3 1 7
2 1 5
1 1 5
2 1 5
17
32
```

```
5 3
3 7 2 2 5
3 2 3
1 3 6
1 2 3
9
```